

1. Loading

The used models can be seen in the first code cell (CHECKPOINTS).

Relevant training and model information can be seen in "Relevant Training Information".

```
In [63]: import sys
from os import path, getcwd
import pathlib

# where everything is stored (data and code)
ROOT = pathlib.Path(getcwd()).parent.absolute()
# code paths
sys.path.append(path.join(ROOT, 'src/models'))
sys.path.append(path.join(ROOT, 'src/exp'))
sys.path.append(path.join(ROOT, 'src/common'))

# data paths
DATA_DIR = path.join(ROOT, 'data/')
# checkpoints of preprocessed data
CHECKPOINTS = {'nf': path.join(DATA_DIR, 'checkpoints/nf/RealNVP_6_mnist_cls-0/05-21'),
               'survae': path.join(DATA_DIR, 'checkpoints/survae/survae_steps-12_scal')}
```

```
In [64]: %reload_ext autoreload
%autoreload 2

# Plotting Functions
from plotting import *
from precomputation.jupyter_imports import *
from misc.common_functions import load_configs

_=%matplotlib inline
```

1.1 Loading Models and Training Data

```
In [65]: # config
configs, MAIN_TEST_CLS = load_configs(CHECKPOINTS, DATA_DIR, verbose=False)
# Data
all_loaders = load_data(configs, verbose=False)
# Precomputed information
all_losses, samples, artificial_images, _ = load_preprocessings(CHECKPOINTS)
```

2. Notation and Definitions

We begin by establishing notation and definitions.

Please refer to this section in case used notation is unclear.

$\mathcal{X}$: Data-Space (e.g. $\mathcal{X} = [0, 1]^{28 \times 28} \subseteq \mathbb{R}^{28 \times 28}$ for mnist)

$\mathcal{Z}$: Latent-Space (e.g. $\mathcal{Z} = \mathbb{R}^{(28 \cdot 28)}$ for MNIST and a Normalizing Flow)

$x \in \mathcal{X}$: x is referred to as image

$z \in \mathcal{Z}$: z is referred to as latent code

$D \subseteq \mathcal{X}$: real world dataset, finite

$D = D_0 \cup \dots \cup D_L$: Partition on the training set, where the index denotes the class and L is the last class (e.g. D_3 for MNIST is the set containing all images with the number "3")

$D_i = D_i^t \cup D_i^e$: Partition on D_i . D_i^t is a training set and D_i^e is a test set (e for evaluation).

D_c^t : main training data set

c : main class on which we train.

X_c : Random variable / random image from which the images in D_c were drawn

Z : Random variable in latent space

$X_c \sim F$: X_c is distributed according to an unknown probability distribution F . Our goal is to approximate the true probability density function (pdf) $p^*(x_c)$.

$Z \sim G$: G is the prior distribution of Z . Usually $G = N(0, 1)$.

$p_Z(z)$: known pdf of G

x, z : a sample / realization drawn from the according distribution

$f_\theta : \mathcal{Z} \rightarrow \mathcal{X}$ Transformation

$f_\theta^{-1} : \mathcal{X} \rightarrow \mathcal{Z}$ Inverse Transformation

θ trainable parameters of f . For simplicity we drop θ .

$p_X(x) = p(x) = p_z(f^{-1}(x)) \cdot V(x, z)$: approximation for p^*

$V(x, z)$: likelihood contribution (see: [1, Section 2])

$\log p(x)$ in bpd = $\frac{\log p_X(x)}{\dim(x) \cdot \log(2)} = \frac{\log_2 p_X(x)}{\dim(x)}$: log likelihood in bits per dimension

$\max_\theta \sum_{x \in D_c^t} \log p_X(x)$: the optimization term during training

This means, for an optimal f , $f(Z) = X \sim F$

We trained two models. A Normalizing Flow (NF) (the RealNVP Flow from [2]) and a Surjective Variational Autoencoder (SurVAE) (the MaxPoolFlow from [2, 1.3]). We sometimes refer to them as f_{NF}^{-1} and f_{NF} or f_{SVAE}^{-1} and f_{SVAE} respectively.

Above each plot, we denote which model was used to compute the seen values.

Note that we evaluate the checkpoint that performed best on the test set D_c^e during training.

Legend Denomination

" c (test)" refers to the test subset D_c^e

" c (train)" refers to the test subset D_c^t

" j " where $j \neq c$ refers to all other the classes which are not used for training. The datasets used for evaluation of the other classes the training sets D_j^t , as it is bigger and not used during training.

"Mean" of j refers to the average log likelihood in bpd: $\frac{1}{|D_j^h|} \sum_{x \in D_j^h} \log p_X(x)$ in bpd, where $h \in \{t, e\}$

We did not use Latex within the plots as it made plotting slow and more unclear.

References:

[1] Nielsen, Didrik, et al. "SurVAE Flows: Surjections to Bridge the Gap between VAEs and Flows." Advances in Neural Information Processing Systems 33 (2020).

[2] Kirichenko, Polina, Pavel Izmailov, and Andrew Gordon Wilson. "Why normalizing flows fail to detect out-of-distribution data." arXiv preprint arXiv:2006.08545 (2020).

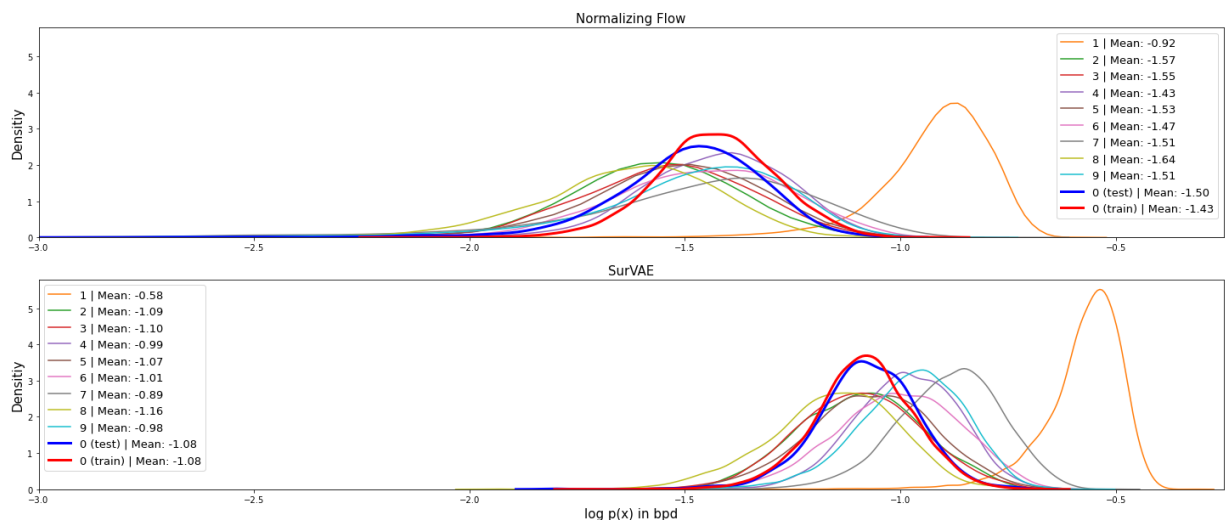
3. KDE plots of the log likelihoods in bpd

We first plot the histogram of $\{\log p_X(x) \text{ in bpd} | x \in D_j\}$ with a kernel density estimation, where j can be seen in the legend.

Note that the x axes are aligned.

We see how both models assign much higher likelihoods to class '1'. We have repeated this experiment with training class '8' and '1' and find the same result: class '1' gets highest likelihood.

```
In [66]: _=show_loss_histograms(all_losses, lim_left=-3, lim_right=-0.25)
```



4. Outlier Datapoints

In this section we look at real world samples from D and plot the ones which obtain lowest and highest likelihoods.

From a visual interpretation, we observe the same as Kirichenko [2], which is: the models seem to have learned a local pixel correlation.

This means that an image with more structure (e.g. one with a '0') can more often fail to match the pixel correlation than one with a simple structure (e.g. a '1', which is often drawn as a pipe). This leads to a lower likelihood. The unmatched pixel correlation is especially visible in the pixelated borders of the number.

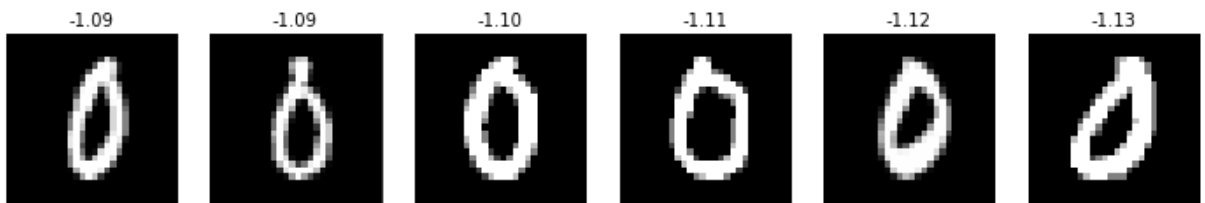
Kirichenko [2] proposes methods to overcome this, but they are either not fully unsupervised or perform poorly.

[2] Kirichenko, Polina, Pavel Izmailov, and Andrew Gordon Wilson. "Why normalizing flows fail to detect out-of-distribution data." arXiv preprint arXiv:2006.08545 (2020).

4.1.1 Images of D_0^e with highest likelihood (descending)

```
In [68]: cols = 6
plot_best_or_worst_samples('nf', [MAIN_TEST_CLS], all_losses, all_loaders, best_imag
```

Title = log p(x) in bpd
Normalizing Flow



```
In [50]: plot_best_or_worst_samples('survae', [MAIN_TEST_CLS], all_losses, all_loaders, best_
```

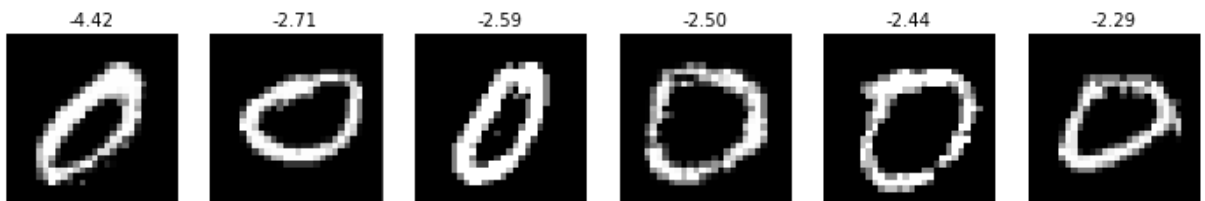
(log p(x) in bpd, pixel mean, class)
SurVAE



4.1.2 Images of main class D_0^e with lowest likelihood (ascending)

```
In [74]: plot_best_or_worst_samples('nf', [MAIN_TEST_CLS], all_losses, all_loaders, best_imag
```

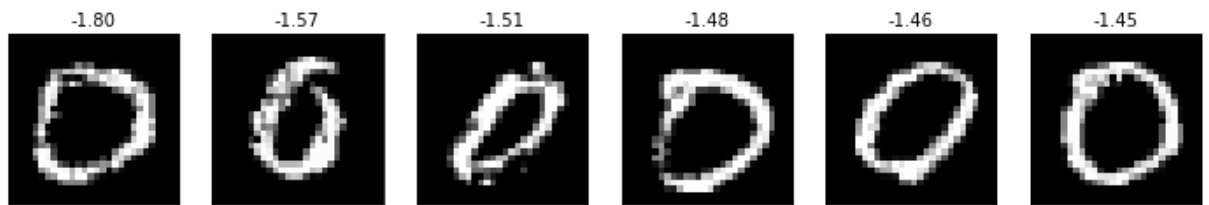
Title = log p(x) in bpd
Normalizing Flow



```
In [73]: plot_best_or_worst_samples('survae', [MAIN_TEST_CLS], all_losses, all_loaders, best_
```

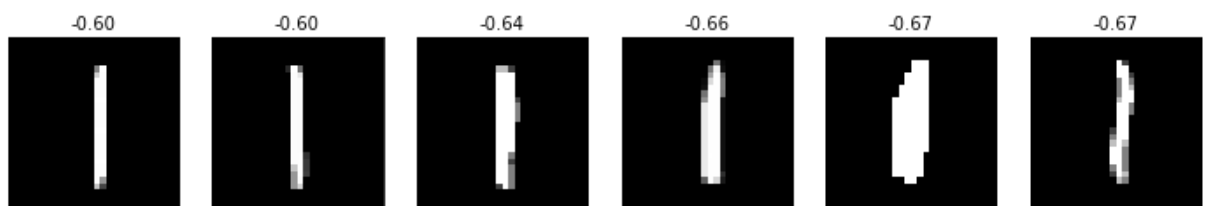
Title = log p(x) in bpd

SurVAE

Images with high likelihood from D_1

In [72]: `plot_best_or_worst_samples('nf', [1], all_losses, all_loaders, best_images=True, amo`

Title = log p(x) in bpd
Normalizing Flow



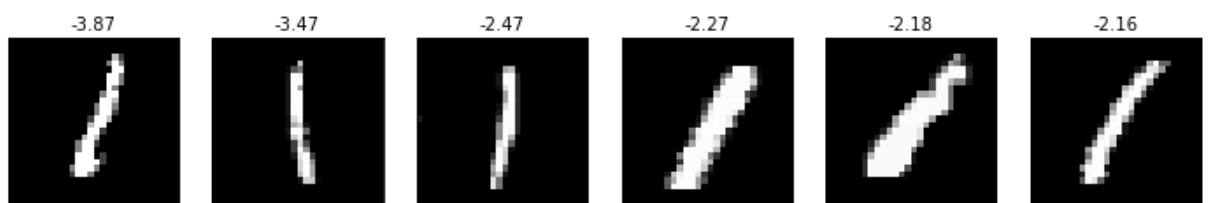
In [71]: `plot_best_or_worst_samples('survae', [1], all_losses, all_loaders, best_images=True,`

Title = log p(x) in bpd
SurVAE

Images with low likelihood from D_1

In [70]: `plot_best_or_worst_samples('nf', [1], all_losses, all_loaders, best_images=False, am`

Title = log p(x) in bpd
Normalizing Flow



In [69]: `plot_best_or_worst_samples('survae', [1], all_losses, all_loaders, best_images=False`

Title = log p(x) in bpd

SurVAE

